

Evaluation

Building agents by building their harness

Lesson 10 of 10 · Production

Where we left off

Lesson 9 made the agent faster and cheaper.

It trimmed results. It chose smaller models.

Is it still **as good**?

How do you know?

You ran it once. It worked.

Then you changed something.

You ran it once. It worked.

Why that's not enough

A model's answers vary.

One run proves little.

A change can break something you didn't try.

Every lesson so far

Each one changed the harness.

Each change could make it **worse** while looking fine.

Evaluation

Run the agent on tasks whose right answer you know, and check the results.

Where does it live?

Evaluation is built on **the whole harness**.

Why the whole harness

You're not testing one primitive.

You're testing what the five do **together**.

From the outside

An evaluation runs the agent like a user would.

Gives it a task.

Looks at what happened.

And it's built from primitives too

- **control flow:** a loop over the cases
- **input:** the task it hands over
- **output:** a command that checks the result
- **model interface:** a model, when the judge is a model

Three parts

1. **tasks** with known outcomes
2. **grading** by code or by model
3. evals **on every change**

Part 1: tasks with known outcomes

A case

A task, and a way to know if it was done right.

Four fields

- `name`
- `setup` : prepare the folder
- `task` : what we ask the agent
- `check` : a shell command

check

A shell command run **after** the agent finishes.

Exit 0 means pass.

Anything else means fail.

Why a shell command

The check doesn't trust the agent's words.

It looks at the **result**.

Did the file change? Does the test pass?

The worked example

`production/10-evaluation/quark.py`

Lesson 9's `quark.py` plus evaluation.

196 lines.

What's new

- `CASES`
- `evaluate(names)`
- a `--eval` flag

Plus a small change to the imports.

The cases

Four of them:

count , fix , rename , remember

count

A question with one right number.

fix

A project with a bug.

`calc.py` subtracts where it should add.

The check runs `test.py`, and checks the assertion is still there.

rename

A change across files.

The check looks at the files.

remember

Lesson 4's semantic memory.

Tell it a fact. Then see if it was **written down**.

Part 2: running them

The key move

`evaluate()` runs `quark.py` **itself**.

As a subprocess.

Once per case.

Why a subprocess

Not a function call.

The real program, from the command line.

Exactly what a person would run.

A fresh folder

Each case runs in a new **temp directory**.

`setup` fills it.

No case sees another's leftovers.

The guard

Lesson 6 asks before commands.

Nobody's there to answer.

So the eval sends `y` fifty times on stdin.

Grading

After the agent ends:

run `check` on the host.

Exit code, pass or fail.

Reading the trace

Lesson 5 pays off again.

The child writes `.quark/traces.jsonl`.

The eval reads **steps** and **tokens** from it.

Pass is not the whole story

Passing and costing **ten times more** is a regression too.

Steps and tokens are measured.

The log

Every run appends to `.quark/evals.jsonl`.

A history.

REGRESSED

If a case **passed last time** and fails now, it's flagged.

REGRESSED.

Why that flag

A failing case may always have failed.

A regressed case is **something you broke**.

Keeping failures

A failed case's temp folder is **kept**.

You can open it and see what the agent did.

The exit code

Any failure: **exit 1**.

A script, or CI, can stop on it.

Where it hooks in

```
if sys.argv[1:2] == ["--eval"]:  
    sys.exit(evaluate(...))
```

Before the program looks for an unfinished session.

Run it

Start in a scratch folder: results are logged there.

```
uv run --project /path/to/building-agents \  
    /path/to/building-agents/production/10-evaluation/quark.py --eval
```

The baseline

4 of 4 pass.

About 18 seconds.

Every future change is compared to this.

Break it on purpose

A copy with `MAX_STEPS = 1`.

The agent can make one model call and no more.

The result

3 of 4.

`fix` fails, flagged **REGRESSED**.

Exit code 1.

Open the kept folder

`calc.py` still subtracts.

The trace shows **one model call**: `cat calc.py test.py` .

The run stopped before the model saw what it read.

A guard did that

Lesson 6's step limit.

An evaluation caught a limit set too low.

Another: trim harder

```
MAX_RESULT = 20 .
```

Lesson 9's trim, set tiny.

Still 4 of 4.

But the cost moved

`fix` went from 3 steps to 4.

Tokens went from about 20k to 27k.

The pass rate hid it. The trace didn't.

What we learned

Two changes, two outcomes.

One broke a case. One passed everything and cost more.

You need the pass rate **and** the cost to see both.

Going further

`quark.py` is one way to do it.

`evaluation.py` is a richer one.

evaluation.py

Built on Lesson 3's `control_flow.py` .

122 lines.

agent()

The loop from `control_flow.py` , as a **function**.

Call it with a task and a variant. Get a result.

Variants

Different ways to run the agent.

Compared **side by side**.

Three of them

- **sonnet**: the baseline
- **haiku**: a smaller model
- **hasty**: the same model, told to use as few commands as it can and read nothing before it edits

Cases with code checks

```
count, fix, log.
```

A shell command decides. Cheap and exact.

Some tasks have no exact answer

"Explain what this function does."

No command can grade that.

Grading by model

`explain` has a **rubric**.

A model reads the answer and the rubric.

It says **PASS** or **FAIL**, with one sentence why.

The judge

A different, stronger model.

```
claude-opus-5-5 .
```

Can you trust the judge?

A judge can be wrong.

calibrate()

Give the judge **three answers whose grade you know**.

Check it agrees.

Before it grades anything real.

Trials

```
TRIALS = 3 .
```

Each case, each variant, three times.

Why three

Answers vary.

One pass might be luck.

Three tells you something.

A pool

```
AT_ONCE = 6 .
```

Whole trials run at the same time.

Running model calls in parallel is control flow, not Lesson 9's tool executor.

The table

Cases down. Variants across.

Passes out of trials.

Steps. Tokens.

worse than

If a variant does **worse than the baseline**, it's called out.

And the exit code is 1.

Run it

```
uv run production/10-evaluation/evaluation.py
```

36 runs.

The result

Every variant passed all 12.

Haiku averaged about **3.8 steps**.

Hasty saved nothing.

A null result

`hasty` wasn't cheaper or faster.

That's a finding.

You'd have guessed otherwise.

Break it on purpose

A copy with a fourth variant: **onestep**.

One step allowed.

The result

3 of 12.

Exit code 1.

Cheaper, because it stopped early

It used fewer tokens.

Because it **gave up**.

Cost alone would say "better".

Always read both

Pass rate and cost.

Either one alone misleads.

A story about a grader

An earlier run: baseline Sonnet got **2 of 3** on `explain` .

That looked like a real weakness.

Read the failures

The judge had answered `**PASS**`.

With markdown stars.

The bug was ours

The parse checked `startswith("PASS")` .

`**PASS**` doesn't start with `PASS` .

The agent was right. The **grader** was wrong.

The fix

Strip the stars and marks first:

```
rstrip("*#` ")
```

calibrate() didn't catch it

It checked the judge's judgment.

Not our reading of it.

A grader has bugs too.

Another fix: the rubric

A word in it was ambiguous.

"returns" or "computes".

Reworded.

The lesson

When an eval fails, **read the failure.**

Before blaming the agent.

What evaluation can be

- **tasks:** a few, or hundreds
- **grading:** code, a model, a person
- **trials:** once, or many
- **when:** on demand, or on every change
- **compared to:** last run, or other variants

Part 3: on every change

The habit

Run it after each change.

Not only when something seems wrong.

The exit code makes it automatic

Exit 1 on failure.

A script or CI runs it on every change.

A change that breaks a case you wrote is caught before it ships.

Where we've been

Lesson 5: you can see what it did.

Lesson 6: it asks first.

Lesson 7: it's boxed.

And

Lesson 8: it survives failure.

Lesson 9: it's fast and cheap.

Lesson 10: you **know** it still works.

Honest limits

- killing it on a timeout can leave a **Docker container**
- each case has its own folder, so cases **share no cache**

What to take away

Rule: evaluation runs the whole harness on tasks with known outcomes, grades by code or model, and runs on every change.

Notice what evaluation never does

It doesn't do any of the five primitives **for** the agent.

It doesn't act as control flow, input, context, model interface, or output **in** the agent.

It stands **outside** and measures.

The whole course

Five primitives.

Six layers that harden them.

One agent, **quark**.

And a way to read any agent

Sort its parts into the five.

Ask what each layer is built on.

Tell the author what doesn't fit.

Back to the course

You've built a harness, and you can measure it.